

Estudo Dirigido 03 — Evolução de Código na Manutenção

INTEGRANTES DO GRUPO:

- Pedro Paiva Ferreira
- Gabriel Nascimento Krepker
- Joao Victor da Costa
- Rafael Henriques Lima
- Gabriel Monteiro
- Mateus Silva Xavier

Repositório: <https://github.com/LuqmanSahaf/LibraryManagementSystem>

Linguagem: Java

Ferramentas: IntelliJ IDEA, PMD 7.13.0, CPD (Copy-Paste Detector)

1. Introdução

A manutenção de software é responsável por mais de 60% do custo total de um projeto ao longo de sua vida útil (Sommerville, 2011). Nesse cenário, manter o código compreensível e evolutivo é tão crítico quanto fazer funcionar.

Refatoração (Fowler, 1999) é a prática de melhorar a estrutura interna do código **sem alterar seu comportamento observável**, com o objetivo de torná-lo mais limpo e fácil de evoluir. Os princípios de **Código Limpo** (Martin, 2008) reforçam essa ideia: nomes intencionais, funções pequenas e focadas, ausência de duplicação e encapsulamento adequado.

Este trabalho analisa um sistema real de biblioteca em Java disponível no GitHub, identifica problemas estruturais utilizando a ferramenta **PMD 7.13.0**, aplica refatorações e compara métricas coletadas automaticamente antes e depois das melhorias.

2. Descrição do Sistema

O sistema **LuqmanSahaf/LibraryManagementSystem** é uma aplicação de console Java que gerencia uma biblioteca universitária. O projeto original possui **16 arquivos Java e 1.738 linhas de código**, sem frameworks externos.

Funcionalidades principais:

- Login de administradores, professores e estudantes
- Cadastro e remoção de usuários e recursos (livros, pacotes de curso, revistas)
- Empréstimo e devolução com prazo configurável por tipo de usuário
- Renovação de recursos emprestados
- Cálculo de multas por atraso (R\$100/dia para livros, R\$500/dia para pacotes de curso)
- Solicitações de recursos ocupados por outros usuários

Arquivos originais e respectivos tamanhos (LOC):

```
Driver.java - 539 linhas - interface de console + logica misturada
Borrower.java - 343 linhas - acumulo de responsabilidades
Library.java - 321 linhas - God Class (WMC=53)
Borrowable.java - 152 linhas - interface com implementacao
Admin.java - 88 linhas
LibraryUser.java - 58 linhas
Book.java / CoursePack.java / Magazine.java
Student.java / Faculty.java / Fine.java
Constants.java / LibraryInfo.java / LibraryResource.java
```

3. Problemas Encontrados

A análise com o PMD 7.13.0 (categorias **design**, **bestpractices**, **codestyle**, **errorprone** e **performance**) retornou **733 warnings**. O CPD detectou **11 blocos de código duplicados**.

#	Code Smell	Onde / Evidência PMD	Impacto na Manutenção
1	God Class	Library.java - GodClass (WMC=53, ATFD=12)	Qualquer mudança exige entender toda a classe
2	Long Method	Driver.loginIO (CC=30, NPath=288, 200+ linhas)	Impossível testar; alto risco a cada alteração
3	Duplicate Code	Borrower: viewIssued/viewIssueDate s/viewDueDates - 9 linhas × 3	Mudar formato exige alterar 3 métodos separados
4	Magic Numbers	Library.updateFines: days*100, days*500 hardcoded	Regras de negócio ocultas; perigoso de alterar

#	Code Smell	Onde / Evidência PMD	Impacto na Manutenção
5	Falta de Encapsulamento	87 campos package-private (CommentDefaultAccess Modifier)	Estado mutável diretamente por qualquer classe
6	Acoplamento Excessivo	101 LawOfDemeter – acesso direto a campos internos	Refatorações locais causam efeito cascata
7	Tipo Legado (Vector)	13 LooseCoupling – Vector<Integer> obsoleto desde Java 1.2	Sincronização desnecessária, API antiquada
8	Singleton Inconsistente	getInstance("LUMS") vs getInstance("LUMS Library")	Bug latente; objetos diferentes em contextos distintos
9	Código Comentado	tryReturn(), tryRenew() com System.out comentados	Código morto sem rastreadibilidade
10	Alta Complexidade	Borrower.tryIssue: NPath=510, CC=14	Difícil testar todas as ramificações

4. Refatorações Aplicadas

A tabela abaixo resume cada refatoração realizada, seguindo o catálogo de Fowler (1999).

Técnica	Problema atacado	O que foi feito	Resultado
Extract Class	God Class Library (WMC=53)	Dividida em RepositorioRecursos, RepositorioUsuarios, AdminService, EmprestimoService	Responsabilidade única por classe (SRP)
Extract Method	loginIO com 200+ linhas (CC=30)	Decomposto em MenuPrincipal, MenuAdmin e MenuUsuario com métodos curtos	Maior método com ~20 linhas; CC máxima = 10
Remove Duplicate Code	3 métodos quase idênticos em Borrower	Unificados em ExibidorDeRecursos.ex ibirRecursosEmprestad os()	CPD: 11 – 3 blocos detectados
Replace Magic Number	days*100, days*500, prazo 15/30 hardcoded	Constantes MULTA_DIARIA e PRAZO_DIAS nos modelos; limites no enum TipoUsuario	Regras de negócio nomeadas e centralizadas

Técnica	Problema atacado	O que foi feito	Resultado
Replace Type Code with Enum	Constantes 1, 2, 3 para tipo de usuário	Enum TipoUsuario com limiteEmprestimos por tipo	Sem magic numbers; segurança de tipos
Encapsulate Field	87 campos package-private	Todos private com getters e métodos de negócio	CommentDefaultAccessModifier zerado
Replace Collection Type	Vector<Integer> obsoleto (13 LooseCoupling)	Substituído por List<Integer> e ArrayList	Sem sincronização desnecessária
Introduce Exception	continue + println para tratamento de erro	IllegalArgumentException / IllegalStateException	Lógica de erro separada da lógica de negócio
Rename Variable/Method	l, u, m, da, iu, bor (ShortVariable × 13)	Renomeados para nomes descritivos em português	Legibilidade sem necessidade de comentários

4.1 Exemplo: Encapsulamento de Campos

No original, **87 campos eram package-private** – qualquer classe acessava e modificava o estado interno diretamente, sem controle.

Antes - campos sem modificador de acesso em LibraryUser.java:

```
// Qualquer classe pode acessar e alterar diretamente
String userName;
String password;
int userID;
int type;
```

Depois - encapsulamento completo em Usuario.java:

```
private final int id;
private final String nomeUsuario;
private final String senha;
private final TipoUsuario tipo;

// Estado alterado apenas via metodos de negocio
public void registrarEmprestimo(int idRecurso) {
    recursosEmprestados.add(idRecurso);
}
```

4.2 Exemplo: Eliminação de Código Duplicado

O CPD detectou **9 linhas idênticas repetidas 3 vezes** em Borrower.java nos métodos viewIssued(), viewIssueDates() e viewDueDates().

Antes - duplicação em Borrower.java (3 métodos quase idênticos):

```
void viewIssued() {
    if (issuedResources.size() == 0) { ... return; }
```

```

Library lib = Library.getInstance("LUMS Library");
for (int i = 0; i < issuedResources.size(); i++) {
    res = (Borrowable)(lib.findResource(issuedResources.get(i)));
    System.out.println(...); // unica linha diferente
}
}
// viewIssueDates() e viewDueDates() com o mesmo padrao

```

Depois - método único em `ExibidorDeRecursos.java`:

```

public void exibirRecursosEmprestados(List<Integer> ids) {
    if (ids.isEmpty()) {
        System.out.println("Nenhum recurso emprestado.");
        return;
    }
    System.out.println("No.\tID --- Recurso --- Emprestimo --- Devolucao");
    for (int i = 0; i < ids.size(); i++) {
        exibirLinhaRecurso(i + 1, ids.get(i));
    }
}

```

4.3 Exemplo: Magic Numbers - Constantes Nomeadas

Antes - números mágicos em `Library.java`:

```

// O que significam 100 e 500?
if (borrowable.type == Constants.BOOK) {
    amount = days * 100;
} else if (borrowable.type == Constants.COURSE_PACK) {
    amount = days * 500;
}
// Prazo hardcoded em tryRenew():
borrowable.renewResource(30); // professor
borrowable.renewResource(15); // estudante

```

Depois - constantes nomeadas em `Livro.java` e `PacoteCurso.java`:

```

public class Livro extends RecursoEmprestavel {
    private static final int PRAZO_ESTUDANTE_DIAS = 15;
    private static final int PRAZO_PROFESSOR_DIAS = 30;
    private static final int MULTA_DIARIA = 100;

    public int getPrazoPara(TipoUsuario tipo) {
        if (tipo == TipoUsuario.PROFESSOR) return PRAZO_PROFESSOR_DIAS;
        return PRAZO_ESTUDANTE_DIAS;
    }
}

```

5. Comparação Antes e Depois

5.1 Métricas Quantitativas (PMD 7.13.0 + CPD)

Os valores abaixo foram coletados automaticamente pelo PMD 7.13.0 e pelo CPD, executados localmente sobre o código-fonte original e sobre o código refatorado.

Métrica	Antes	Depois	Redução
Warnings PMD total	733	238	-67,5%
Blocos duplicados (CPD)	11	3	-72,7%
GodClass	1	0	-100%
TooManyMethods	2	0	-100%
Complexidade Ciclomática máxima	30	10	-67%
LawOfDemeter	101	1	-99%
Campos sem encapsulamento (CommentDefaultAccessModifier)	87	0	-100%
LooseCoupling (Vector obsoleto)	13	0	-100%
ForLoopCanBeForeach	12	0	-100%
SystemPrintln espalhados	108	30	-72%
LOC total	1.738	788	-55%
Classes / arquivos Java	16	16	—

5.2 Comparação Qualitativa

Critério	Antes	Depois
Legibilidade	Variáveis de 1-2 letras; lógica de negócio misturada com UI	Nomes descritivos em português; responsabilidades separadas
Complexidade	Driver.loginIO com CC=30 e 200+ linhas gerenciando tudo	Camadas model / service / ui; maior método com ~20 linhas
Duplicação	9 linhas repetidas 3x em Borrower; padrões de busca duplicados em Driver	ExibidorDeRecursos centraliza a exibição; 3 blocos CPD residuais
Organização	1 God Class (Library) gerencia usuários, recursos e multas	16 classes com responsabilidade única distribuída em pacotes
Manutenibilidade	Adicionar tipo de usuário exige alterar 4+ classes	Nova entrada no enum TipoUsuario é suficiente
Acoplamento	101 violações LawOfDemeter; campos públicos acessados diretamente	LawOfDemeter: 101 - 1; estado acessado via métodos
Coesão	Library mistura domínio, persistência, UI e regras de multa	Cada classe trata de um único conceito de domínio
Tamanho dos métodos	loginIO com 200+ linhas; tryIssue com 60 linhas	Métodos com foco único; maior com ~20 linhas
Testabilidade	Impossível isolar - depende do Singleton Library global	Injeção de dependência via construtor permite testes unitários

6. Conclusão

6.1 Principais Melhorias Obtidas

- Redução de 67,5% dos warnings PMD (733 - 238), eliminando todos os alertas de design estrutural.
- Redução de 72,7% nos blocos de duplicação CPD (11 - 3), com 3 blocos residuais de menor impacto.
- God Class Library (WMC=53) substituída por 4 classes com responsabilidade única.
- Complexidade ciclomática máxima reduzida de 30 para 10 - métodos menores e mais testáveis.
- 87 campos package-private tornados private, com acesso controlado por métodos de negócio.
- Substituição do Vector obsoleto por List/ArrayList, eliminando sincronização desnecessária.
- Correção do bug latente de inconsistência no Singleton ("LUMS" vs "LUMS Library").

6.2 Dificuldades Encontradas

- Garantir que o comportamento externo permanecesse idêntico sem testes automatizados - feito por inspeção manual.
- Decidir o ponto de parada da refatoração sem torná-la excessivamente acadêmica.
- Separar responsabilidades em classes distintas sem criar acoplamento desnecessário entre elas.

6.3 Impactos na Manutenção

A versão refatorada apresenta melhorias mensuráveis: adicionar um novo tipo de usuário agora exige apenas uma entrada no enum **TipoUsuario** - antes exigia alterar pelo menos 4 classes. Alterar a multa de livros é feito em uma constante em **Livro.java** - antes era um número mágico em Library.java. A estrutura em camadas facilita a localização de qualquer regra de negócio.

6.4 Percepção sobre Qualidade de Software

O trabalho deixou claro que **qualidade de software não é estética** - é uma propriedade econômica. Um código limpo custa menos para evoluir, corrigir e entender. As

ferramentas PMD e CPD tornaram visíveis problemas que passariam despercebidos em revisão manual, e a comparação numérica transformou uma avaliação subjetiva em evidência técnica verificável.